

Python Backend Engineering Assessment

1. Purpose of the Assessment

This assessment is designed to evaluate a candidate's ability to design, implement, and reason about production-grade Python backend systems. The focus is on correctness, concurrency handling, performance awareness, clean architecture, and testability.

2. Assessment Details

- Seniority Level: Mid to Senior Backend Engineer
- Time Expectation: 4–6 hours
- Programming Language: Python 3.10 or higher
- Framework: FastAPI / Django / Flask (candidate choice)
- Database: PostgreSQL preferred (SQLite acceptable for demonstration)
- Testing Framework: Pytest (mandatory)

3. Problem Statement

You are required to build a lightweight Data Ingestion and Aggregation Service. The service should accept incoming events, ensure idempotent and concurrency-safe storage, aggregate events efficiently, and expose APIs to query both raw and aggregated data.

4. Data Model Requirements

Event Entity:

- event_id (string, globally unique)
- tenant_id (string)
- source (string, e.g. web, mobile, device)
- event_type (string, e.g. click, view, error)
- timestamp (UTC datetime)
- payload (JSON object)



- created_at (server-generated timestamp)

Aggregate Entity:

- tenant_id (string)
- bucket_start (datetime)
- bucket_size (minute or hour)
- source (optional dimension)
- event_type (optional dimension)
- count (integer)
- first_seen (datetime)
- last_seen (datetime)

5. API Requirements

POST /events

- Accepts a single event
- Must be idempotent based on event_id
- Duplicate submissions must not create duplicate records

POST /events/bulk

- Accepts up to 5,000 events in a single request
- Must be optimized for throughput and memory usage
- Oversized or invalid payloads must be rejected gracefully

GET /events

- tenant_id is mandatory
- Optional filters: source, event_type, from, to
- Pagination and stable sorting are required

GET /metrics

- Returns aggregated counts
- Supports minute or hour level bucketing
- Optional dimensions: source, event_type
- Must not perform naive in-memory aggregation

GET /health – basic liveness check

GET /ready – readiness check including database connectivity

6. Python Engineering Expectations

Concurrency and Idempotency

The implementation must safely handle concurrent requests without race conditions. Acceptable approaches include database constraints with transactional handling or explicit locking strategies.

Asynchronous I/O

At least one major code path (bulk ingestion, aggregation, or I/O) must demonstrate effective asynchronous programming without blocking calls.

Background Processing

Implement a background task or worker (or CLI command) to process event aggregation. The process must be incremental and idempotent.

7. Performance & Optimization

Candidates must demonstrate performance awareness by applying batching, indexing, or query optimization. At least one profiling or benchmarking insight should be documented.

8. Security & Abuse Controls

- Input size validation and payload limits



- Timestamp sanitization and UTC enforcement
- Basic rate-limiting strategy (implementation or explanation)

9. Testing Requirements

Automated tests using Pytest must cover:

- Idempotent event ingestion
- Bulk ingestion validation
- Aggregation correctness
- Concurrency safety
- Time window edge cases

10. Submission Deliverables

- Source code repository or zip archive
- README with setup, run, and test instructions
- Architecture Notes (1–2 pages) explaining design decisions

11. Evaluation Criteria

- Correctness and data consistency
- Depth of Python knowledge
- Concurrency handling
- Performance considerations
- Test quality and coverage
- Code readability and structure